All-pairs shortest paths: solutions

25-E01 The second step of two different DPs

Derivation

Rows of the at-most-two-edge $L^{(2)}$ are$[0,4,2,10],[\infty,0,-2,1],[\infty,4,0,3],[\infty,1,-1,0]$. Rows of $D^{(2)}$, allowing intermediates$\{1,2\}$, are$[0,4,2,10],[\infty,0,-2,6],[\infty,\infty,0,3],[\infty,1,-1,0]$.

Entry $L^{(2)}[2,4]=1$ uses$2\to3\to4$, whose intermediate3 is forbidden in $D^{(2)}$, so $D^{(2)}[2,4]=6$. Entry $L^{(2)}[3,2]=4$ uses$3\to4\to2$, whose intermediate4 is also forbidden, so $D^{(2)}[3,2]=\infty$.

The four1-to 4 candidates are k1:$0+\infty=\infty$, k2:$4+6=10$, k3:$7+3=10$, k4:$\infty+0=\infty$. Their minimum is 10. A three-edge budget allows $1\to2\to3\to4$, so $L^{(3)}[1,4]=4-2+3=5$, also obtained as $L^{(2)}[1,3]+W[3,4]=2+3$.

Conclusion and checks

The second step restricts edge count in one DP and intermediate vertices in the other. For n=4 without negative cycles, $L^{(3)}[1,4]=D^{(4)}[1,4]=5$, without equality of the second-stage matrices.

25-E02 Negative diagonals and locally affected pairs

Derivation

Cycle $2\to3\to2$ costs$-3+1=-2$. Sources able to enter are$\{1,2,3\}$; destinations reachable from it are$\{2,3,4\}$. Their Cartesian product gives 9 affected pairs. Complete rows are$[0,-\infty,-\infty,-\infty,\infty],[\infty,-\infty,-\infty,-\infty,\infty],[\infty,-\infty,-\infty,-\infty,\infty],[\infty,\infty,\infty,0,\infty]$, $[\infty,\infty,\infty,\infty,0]$.

Although1 can enter the cycle, no exit returns to 1. Its best self-route is the zero-length path, so $\delta(1,1)=0$. Entering the cycle, repeating r times, and leaving for 4 costs $2-3-2r+4=3-2r$, so $\delta(1,4)=-\infty$. Vertex 4 has no outgoing edge, so $\delta(4,1)=\infty$. Isolated5 retains its zero-length path and $\delta(5,5)=0$.

Conclusion and checks

A negative D[2,2] witnesses a negative closed walk, not a minimum. Each further repetition reduces cost by 2, so $\delta(2,2)=-\infty$. Finite0, unreachable∞, and unbounded-below$-\infty$ are distinct meanings.

All-pairs shortest paths: solutions (continued)

25-E03 Johnson's cost model

Derivation

Floyd performs $n^3=10^9$ triple-index updates. All-zero potential computation scans n−1 rounds and a test, with leading $nm=5\times10^6$ edge operations plus O(n) initialization. Repeated binary Dijkstra gives $n(m+n)\log_2 n\approx1000\cdot6000\cdot9.966\approx5.98\times10^7$. Fibonacci gives $nm+n^2\log_2 n\approx5\times10^6+9.966\times10^6\approx1.50\times10^7$. These use different queue guarantees and cannot share formulas interchangeably.

Full output has $n^2=10^6$ distances and requires $\Theta(n^2)$ resident space at one word per entry. Johnson's graph and per-source arrays additionally use O(n+m). With original m=0, the augmented graph still has n arcs and n+1 vertices. Generic Bellman–Ford has bound O((n+1)n), not zero. The all-zero original-graph version can instead return h=0 in O(n).

Conclusion and checks

Fibonacci's total bound is $O(nm+n^2\log(n+1))$; binary's is $O(n(m+n)\log(n+1))$. Even an edgeless graph requires initialization and output. These expressions are not seconds or measured speed ratios.

25-E04 Potential and distance restoration

Derivation

Start h=(0,0,0,0). Round 1 lowers h[3] to−2 via2→3; rounds 2,3 are unchanged. Reweighted edges in input order are 1→2:4,1→3:9,2→3:0,2→4:6,3→4:1,4→2:1.

From 1, start(0,∞,∞,∞). Extract 1 to get(0,4,9,∞). Extract 2: candidate3=4+0=4<9 and candidate4=4+6=10, giving(0,4,4,10). Extract 3:4+1=5<10, giving(0,4,4,5). Extract 4:the candidate for 2 is 5+1=6≥4, so unchanged.

Restore δ(u, v)=δHat(u, v)−h[u]+h[v]. Source 1 gives(0,4,4−2,5)=(0,4,2,5). For 3→2, the reweighted distance 2 restores as2−(−2)+0=4. For 4→3, the value 1 restores as1−0−2=−1.

Conclusion and checks

Route 1→2→3→4 costs 4−2+3=5 originally and 4+0+1=5 after reweighting. Its endpoint-potential adjustment happens to be0. The adjustment for 1→3 is+2, so numeric distances are not always unchanged.

All-pairs shortest paths: solutions (continued)

25-E05 Both directions of nonnegative reweighting

Derivation

If a finite h makes all $\hat{w}$ nonnegative, summing around any cycle cancels potentials, giving original cycle weight=$\Sigma\hat{w}\geq 0$. This necessity does not require h to be a source-distance function.

Conversely, with no negative cycle, add an outgoing-only super-source and zero arcs. The augmented graph remains free of negative cycles, all vertices are reachable, and simple shortest paths give finite $h(v)=\delta(s, v)$. Appending $u\to v$ to a shortest route to u gives $h(v)\leq h(u)+w(u, v)$, equivalent to $\hat{w}(u, v)\geq 0$.

For the counterexample, use direct$1\to 4$:6 versus$1\to 2$:2,$2\to 4$:2. Two edges originally cost $4<6$. Adding 3 per edge produces direct$9<10$, reversing the route order. Potential increments telescope to $h[1]-h[4]$ for both routes, so their comparison remains unchanged.

Conclusion and checks

Finite nonnegative reweighting potentials exist exactly when there are no negative cycles. Preservation depends on a constant endpoint difference after telescoping, not simply on increasing edge weights.

25-E06 Predecessors with a zero cycle

Derivation

Source 1 has optimal labels 1:(0,0),2:(0,1),3:(1,1). Direct$1\to 3$ has(1,1), while going via2 has(1,2), so the direct route wins. Set parent[2]=1, parent[3]=1, parent[1]=NIL. Reverse chain$3\leftarrow 1$ has one edge, cost 1.

Cost tightness alone permits parent[1]=2, parent[2]=1, because both zero edges preserve distance 0. This cycles and violates source-parent NIL. Even preserving source NIL does not suffice generally: add source s with zero edges to u, v and zero $u\leftrightarrow v$. Choosing parent[u]=v, parent[v]=u still cycles. The hops condition excludes this by requiring a decrease of exactly one per parent step.

With only edge $1\to 2$:1, initial $\Pi[1,2]=1$. At k=2, the candidate $D[1,2]+D[2,2]=1+0$ ties the old value. Updating on equality assigns $\Pi[1,2]\leftarrow\Pi[2,2]$=NIL and destroys reconstruction.

All-pairs shortest paths: solutions (continued)

Conclusion and checks

Minimum-hop tie-breaking supplies a decreasing integer measure proving acyclicity and termination. Standard Floyd predecessors update on strict improvement and retain old choices on ties. Existence of a shortest path does not validate arbitrary parent choices.

25-E07 Associativity and loop order

Derivation

$((A \otimes B) \otimes C)[i,j] = \min_q(\min_p(A[i,p] + B[p,q]) + C[q,j]) = \min_{p,q}(A[i,p] + B[p,q] + C[q,j])$. Reordering finite minima gives $\min_p(A[i,p] + \min_q(B[p,q] + C[q,j])) = (A \otimes (B \otimes C))[i,j]$. Finite indices and the distribution of addition over minimum justify the reordering.

For n=6 without negative cycles, a simple shortest path has at most5 edges. Successive W, W^2, W^4, W^8 represent budgets1,2,4,8. Budget 8 covers5, and a larger budget introduces no cheaper negative-cycle walk, so W^8=W^5 is the shortest-distance matrix. Negative cycles invalidate that equality; increasing a fixed budget can keep lowering costs.

In chain1→2→4→3, when i=1, j=3, the k=2 candidate needs still-infinite D[2,3], and k=4 needs still-infinite D[1,4]. The entry remains∞. Later j=4 computes D[1,4]=2 using k=2, but never revisits j=3, missing the length3 route. Correct outer k guarantees complete availability of all preceding intermediate vertices. Zero diagonal values keep row/column k unchanged and justify in-place updates.

Conclusion and checks

Associativity groups complete min-plus matrix products. It does not justify arbitrary rearrangement of Floyd's single-pass loops. The proofs manage distinct states.

All-pairs shortest paths: solutions (continued)

Verification code appendix: all_pairs_checks.py

```
"""APSP arithmetic and witnesses checked against exhaustive simple paths/cycles."""
from math import inf
from itertools import product
from heapq import heappush,heappop
import random

SAMPLE={0:[(1,4),(2,7)],1:[(2,-2),(3,6)],2:[(3,3)],3:[(1,1)]}


def matrix(graph):
    n=len(graph);w=[[0 if i==j else inf for j in range(n)] for i in range(n)]
    for u,neighbors in graph.items():
        for v,cost in neighbors:w[u][v]=min(w[u][v],cost)
    return w


def minplus(a,b):
    n=len(a)
    return [[min((a[i][k]+b[k][j] for k in range(n)),default=inf)
             for j in range(n)] for i in range(n)]


def edge_budget(graph):
    n=len(graph);w=matrix(graph);out=[[[0 if i==j else inf for j in range(n)] for i in
range(n)]]
    for _ in range(max(0,n-1)):out.append(minplus(out[-1],w))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

All-pairs shortest paths: solutions (continued)

```
    return out


def squaring(graph):
    n=len(graph);a=matrix(graph);budget=1
    while budget<n-1:a=minplus(a,a);budget*=2
    return a


def floyd(graph,inplace=True):
    n=len(graph);d=matrix(graph);stages=[[row[:] for row in d]]
    for k in range(n):
        old=d if inplace else [row[:] for row in d]
        for i in range(n):
            for j in range(n):d[i][j]=min(old[i][j],old[i][k]+old[k][j])
        stages.append([row[:] for row in d])
    return d,stages


def closure(graph):
    n=len(graph);reach=[[i==j or any(v==j for v,_ in graph[i]) for j in range(n)] for i in
range(n)]
    for k in range(n):
        for i in range(n):
            for j in range(n):reach[i][j]=reach[i][j] or (reach[i][k] and reach[k][j])
    return reach
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All-pairs shortest paths: solutions (continued)

```
def classify(graph,d):
    n=len(graph);reach=closure(graph)
    return [[-inf if any(d[k][k]<0 and reach[i][k] and reach[k][j] for k in range(n))
             else d[i][j] for j in range(n)] for i in range(n)]


def lex_floyd(graph):
    n=len(graph);d=[[(0,0) if i==j else (inf,inf) for j in range(n)] for i in range(n)]
    for u,neighbors in graph.items():
        for v,cost in neighbors:d[u][v]=min(d[u][v],(cost,1))
    for k in range(n):
        for i in range(n):
            for j in range(n):
                candidate=(d[i][k][0]+d[k][j][0],d[i][k][1]+d[k][j][1])
                d[i][j]=min(d[i][j],candidate)
    return d


def predecessor_from_lex(graph,values):
    n=len(graph);parents=[[None]*n for _ in range(n)]
    for source in range(n):
        for u,neighbors in graph.items():
            for v,cost in neighbors:
                if v!=source and values[source][u][0]<inf and values[source][v]==(
                    values[source][u][0]+cost,values[source][u][1]+1):
                    parents[source][v]=u
    return parents


def johnson(graph):
    n=len(graph);h=[0]*n;trace=[h[:]]
    for _ in range(max(0,n-1)):
        old=h[:]
        for u,neighbors in graph.items():
            for v,cost in neighbors:h[v]=min(h[v],old[u]+cost)
        trace.append(h[:])
    if any(h[v]>h[u]+cost for u in graph for v,cost in graph[u]):return None,h,trace
    adjusted={u:[(v,cost+h[u]-h[v]) for v,cost in neighbors] for u,neighbors in
graph.items()}
    assert all(cost>=0 for neighbors in adjusted.values() for _,cost in neighbors)
    result=[];dijkstra_traces=[]
    for source in range(n):
        d=[inf]*n;d[source]=0;heap=[(0,source)];settled=[]
        while heap:
            cost,u=heappop(heap)
            if cost!=d[u]:continue
            settled.append((u,cost))
            for v,weight in adjusted[u]:
```

All-pairs shortest paths: solutions (continued)

```
                if cost+weight<d[v]:
                    d[v]=cost+weight;heappush(heap,(d[v],v))
        result.append([value-h[source]+h[v] if value<inf else inf for v,value in
enumerate(d)])
        dijkstra_traces.append(settled)
    return result,h,trace,dijkstra_traces


def oracle(graph):
    # Enumerate simple directed cycles and simple paths, no dynamic-programming state.
    n=len(graph);cycle_vertices=set()
    for start in range(n):
        def cycles(u,seen,cost):
            for v,weight in graph[u]:
                if v==start:
                    if cost+weight<0:cycle_vertices.update(seen)
                elif v not in seen:cycles(v,seen|{v},cost+weight)
        cycles(start,{start},0)
    reachable=[[False]*n for _ in range(n)]
    answer=[[inf]*n for _ in range(n)];hops=[[inf]*n for _ in range(n)]
    for start in range(n):
        def paths(u,seen,cost,length):
            reachable[start][u]=True
            if (cost,length)<(answer[start][u],hops[start][u]):
                answer[start][u]=cost;hops[start][u]=length
            for v,weight in graph[u]:
                if v not in seen:paths(v,seen|{v},cost+weight,length+1)
        paths(start,{start},0,0)
    for i in range(n):
        for j in range(n):
            if any(reachable[i][k] and reachable[k][j] for k in cycle_vertices):answer[i]
[j]=-inf
    return answer,hops,reachable


def verify(graph):
    answer,hops,reachable=oracle(graph);raw,stages=floyd(graph)
    assert classify(graph,raw)==answer
    buffered,_=floyd(graph,False);assert classify(graph,buffered)==answer
    assert closure(graph)==reachable
    result=johnson(graph)
    negative=any(answer[i][i]==-inf for i in graph)
    assert (result[0] is None)==negative
    if not negative:
        assert raw==buffered==answer==edge_budget(graph)[-1]==squaring(graph)==result[0]
        values=lex_floyd(graph)
        assert values==[[(answer[i][j],hops[i][j]) for j in graph] for i in graph]
        parents=predecessor_from_lex(graph,values)
        for source in graph:
            for target in graph:
                if answer[source][target]==inf:assert parents[source][target] is
None;continue
```

All-pairs shortest paths: solutions (continued)

```
                v=target;cost=0;path=[v]
                while v!=source:
                    u=parents[source][v];assert u is not None
                    cost+=next(weight for w,weight in graph[u] if w==v)
                    v=u;path.append(v);assert len(path)<=len(graph)
                assert cost==answer[source][target] and len(path)-1==hops[source][target]
    return not negative


def main():
    total=0;safe=0
    for n in range(4):
        edges=[(u,v) for u in range(n) for v in range(n) if u!=v]
        for weights in product([None,-2,0,3],repeat=len(edges)):
            graph={u:[] for u in range(n)}
            for (u,v),weight in zip(edges,weights):
                if weight is not None:graph[u].append((v,weight))
            safe+=verify(graph);total+=1
    rng=random.Random(25)
    for _ in range(1200):
        n=rng.randrange(1,7)
        graph={u:[(v,rng.randrange(-4,7)) for v in range(n) if rng.random()<.28] for u in
range(n)}
        safe+=verify(graph);total+=1
    verify(SAMPLE)
    expected=[[0,4,2,5],[inf,0,-2,1],[inf,4,0,3],[inf,1,-1,0]]
    assert floyd(SAMPLE)[0]==expected
    assert edge_budget(SAMPLE)[2]==[[0,4,2,10],[inf,0,-2,1],[inf,4,0,3],[inf,1,-1,0]]
    j=johnson(SAMPLE);assert j[1]==[0,0,-2,0]
    assert j[3][0]==[(0,0),(1,4),(2,4),(3,5)]
    # Wrong i,j,k order fails on this 1->2->4->3 chain, even without negative edges.
    chain={0:[(1,1)],1:[(3,1)],2:[],3:[(2,1)]};wrong=matrix(chain)
    for i in range(4):
        for j in range(4):
            for k in range(4):wrong[i][j]=min(wrong[i][j],wrong[i][k]+wrong[k][j])
    assert wrong[0][2]==inf and floyd(chain)[0][0][2]==3
    isolated_negative={0:[(1,2)],1:[(2,-3)],2:[(1,1),(3,4)],3:[],4:[]}
    assert classify(isolated_negative,floyd(isolated_negative)[0])==[
        [0,-inf,-inf,-inf,inf],[inf,-inf,-inf,-inf,inf],
        [inf,-inf,-inf,-inf,inf],[inf,inf,inf,0,inf],[inf,inf,inf,inf,0]]
    zero_cycle={0:[(1,0),(2,1)],1:[(0,0),(2,1)],2:[]};verify(zero_cycle)
    print(f'PASS APSP_weighted_graphs={total} without_negative_cycles={safe} '
          'Floyd_inplace_and_buffered=minplus=Johnson=independent_simple_paths '
          'negative_pairs=enumerated_simple_cycles lex_predecessors_decrease_hops=True '
          'wrong_ijk_counterexample=True sample_potential=0,0,-2,0 sample_restored_1to4=5')


if __name__=='__main__':main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

All-pairs shortest paths: solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study